

REST

RESTful Providers

Refer to the *rest-social-network* example (labs/rest-social-network):

The **post-service** manages posts on a social network. It does not store details about users. The model class is reported below:

```
@AllArgsConstructor
@NoArgsConstructor
@RequiredArgsConstructor
@Data
@Entity
public class Post {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NonNull @EqualsAndHashCode.Include private String userUUID;
    @NonNull @EqualsAndHashCode.Include private LocalDateTime timestamp;
    @NonNull private String content;
}
```

The REST controller handles incoming HTTP requests and responds with the appropriate data. It supports two key endpoints:

- GET /posts → Returns all posts.
- GET /posts/{userUUID} → Returns all posts created by a specific user.

```
@RestController
@RequestMapping("/posts")
public class PostController {
    PostRepository postRepository;

    public PostController(PostRepository postRepository) {
        this.postRepository = postRepository;
    }

    @GetMapping("/{userUUID}")
    public Iterable<PostDTO> findByUuid(@PathVariable String userUUID) {
        Iterable<Post> foundPosts = postRepository.findByUserUUID(userUUID);
        return mapToDTO(foundPosts);
    }

    @GetMapping
    public Iterable<PostDTO> findAll() {
        Iterable<Post> foundPosts = postRepository.findAll();
        return mapToDTO(foundPosts);
    }

    private Iterable<PostDTO> mapToDTO(Iterable<Post> posts) {
        return StreamSupport.stream(posts.spliterator(), false)
            .map(p -> new PostDTO(p.getUserUUID(), p.getTimestamp(), p.getContent()))
            .collect(Collectors.toList());
    }
}
```

RESTful Consumers

The **user-service** manages data about users. It does expose the following endpoints:

- GET /users → Returns all users (only local details).
- GET /users/{userUUID} → Returns local details and all posts of a specific user (**need to communicate here!**).

```

@RestController
@RequestMapping("/users")
public class UserController {
    UserRepository userRepository;
    PostIntegration postIntegration;

    public UserController(UserRepository userRepository, PostIntegration postIntegration) {
        this.userRepository = userRepository;
        this.postIntegration = postIntegration;
    }

    @GetMapping
    public Iterable<UserDTO> findAll() {
        Iterable<UserModel> foundUsers = userRepository.findAll();
        return mapToDTO(foundUsers);
    }

    @GetMapping("/{userUUID}")
    public UserDTO findByUuid(@PathVariable String userUUID) {
        Optional<UserModel> optionalUserModel = userRepository.findByUserUUID(userUUID);
        optionalUserModel.orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
        UserDTO userDTO = mapToDTO(optionalUserModel.get());

        // communication here!
        Iterable<PostDTO> posts = postIntegration.findbyUserUUID(userUUID);

        for (PostDTO postDTO : posts) {
            userDTO.getPosts().add(postDTO);
        }

        return userDTO;
    }

    private UserDTO mapToDTO(UserModel user) {
        return new UserDTO(
            user.getUserUUID(),
            user.getNickname(),
            user.getBirthDate()
        );
    }

    private Iterable<UserDTO> mapToDTO(Iterable<UserModel> users) {
        return StreamSupport.stream(users.spliterator(), false)
            .map(u -> new UserDTO(u.getUserUUID(), u.getNickname(), u.getBirthDate()))
            .collect(Collectors.toList());
    }
}

```

The communication part is entirely managed by a dedicated bean named *PostIntegration*. Connection data describing the network location of the service to be consumed (in this case, the **post-service**) are externalized to the **application.yml** file and fetched with the **@Value** annotation.

```

@Component
public class PostIntegration {
    String postServiceHost;
    int postServicePort;

    public PostIntegration(
        @Value("${app.post-service.host}") String postServiceHost,
        @Value("${app.post-service.port}") int postServicePort) {
        this.postServiceHost = postServiceHost;
        this.postServicePort = postServicePort;
    }
}

```

```
public Iterable<PostDTO> findbyUserUUID(String userUUID) {  
    String url = "http://" + postServiceHost + ":" + postServicePort + "/posts" + "/" + userUUID;  
    RestClient restClient = RestClient.builder().build();  
    return restClient.get()  
        .uri(url)  
        .retrieve()  
        .body(new ParameterizedTypeReference<>() {});  
}  
}
```

Trying out the messaging system

```
mvn clean package -Dmaven.test.skip=true  
docker compose up --build --detach
```

```
curl -X GET http://localhost:8080/posts | jq
```

```
curl -X GET http://localhost:8081/users | jq
```

```
curl -X GET http://localhost:8081/users/171f5df0-b213-4a40-8ae6-fe82239ab660 | jq
```

Resources

- <https://www.baeldung.com/spring-boot-restclient>